TP5 Android Compose
Système d'Authentification

ISET Gafsa - Développement Mobile Natif

1 Énoncé du TP

Objectif

Créer une application Android avec Jetpack Compose implémentant un système d'authentification (signup/login) avec validation des champs et navigation entre écrans.

Exercice 1 : Questions de compréhension

1. Qu'est-ce que Jetpack Compose et en quoi diffère-t-il du système de vues XML traditionnel ?
2. Expliquez le concept de *state hoisting* dans Compose.
3. Qu'est-ce qu'un `ViewModel` dans l'architecture MVVM ?
4. Comment gère-t-on la navigation entre écrans dans Compose ?
5. Quelle est la différence entre `LaunchedEffect` et `remember` ?

Exercice 2 : Interface Signup

Créer un écran d'inscription avec les champs suivants :
— Nom complet
— Email
— Mot de passe
— Confirmation du mot de passe
— Bouton "S'inscrire"

Exercice 3 : Validation des champs

Implémenter la validation en temps réel :
— Email valide (format standard)
— Mot de passe d'au moins 6 caractères
— Confirmation identique au mot de passe
— Affichage des erreurs sous chaque champ

Exercice 4 : Interface Login

Créer un écran de connexion avec :
— Champ email
— Champ mot de passe
— Bouton "Se connecter"
— Lien vers l'écran d'inscription

Exercice 5 : Écran d'accueil

Créer un écran d'accueil qui affiche :
— Message de bienvenue avec le nom de l'utilisateur
— Bouton de déconnexion

Contraintes techniques

— Utiliser Jetpack Compose
— Aucune base de données (simulation en mémoire)
— Architecture MVVM recommandée
— Gestion d'état avec State et ViewModel

2 Correction du TP

Réponses aux questions

1. **Jetpack Compose** est un toolkit de déclaration d'UI moderne qui utilise des fonctions Kotlin au lieu du XML. Il offre un développement plus intuitif et moins de code boilerplate.
2. Le **state hoisting** consiste à remonter l'état vers le composant parent pour rendre les composants plus réutilisables et testables.
3. Un `ViewModel` conserve les données liées à l'UI pendant les changements de configuration et suit le cycle de vie des composants.
4. La navigation se gère avec le composant `NavHost` et le contrôleur `NavController`.
5. `remember` conserve une valeur pendant la recomposition, `LaunchedEffect` exécute des suspending functions lors d'événements du cycle de vie.

Création du projet dans Android Studio

Étape 1 : Nouveau projet

1. Ouvrir **Android Studio**
2. Cliquer sur **"New Project"**
3. Choisir le template **"Empty Activity"**
4. Cliquer sur **"Next"**

Étape 2 : Configuration du projet

— **Name** : AuthApp
— **Package name** : com.example.authapp
— **Save location** : Choisir votre dossier de projet
— **Language** : Kotlin
— **Minimum SDK** : API 21 (Android 5.0)
— Cochez **"Use legacy android.support libraries"**

Étape 3 : Structure du projet

```
app/
|-- src/main/java/com/example/authapp/
|    |-- MainActivity.kt
|    |-- AuthViewModel.kt
|    |-- screens/
|    |    |-- LoginScreen.kt
|    |    |-- SignUpScreen.kt
|    |    \-- HomeScreen.kt
|    \-- data/
|         \-- User.kt
|
|-- res/
|    |-- values/
|    |    |-- strings.xml
|    |    \-- themes.xml
|    \-- manifest/
|         \-- AndroidManifest.xml
|
\-- build.gradle (Module :app)
```

Structure des fichiers

Configuration des dépendances

Modifier le build.gradle (Module :app)

```kotlin
dependencies {
    implementation("androidx.core:core-ktx:1.12.0")
    implementation("androidx.lifecycle:lifecycle-runtime-ktx:2.7.0")
    implementation("androidx.activity:activity-compose:1.8.2")
    implementation(platform("androidx.compose:compose-bom:2023.08.00"))
    implementation("androidx.compose.ui:ui")
    implementation("androidx.compose.ui:ui-graphics")
    implementation("androidx.compose.ui:ui-tooling-preview")
    implementation("androidx.compose.material3:material3")
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.7.0")
    implementation("androidx.navigation:navigation-compose:2.7.5")

    testImplementation("junit:junit:4.13.2")
    androidTestImplementation("androidx.test.ext:junit:1.1.5")
    androidTestImplementation("androidx.test.espresso:espresso-core:3.5.1")
    androidTestImplementation(platform("androidx.compose:compose-bom:2023.08.00"))
    androidTestImplementation("androidx.compose.ui:ui-test-junit4")
    debugImplementation("androidx.compose.ui:ui-tooling")
    debugImplementation("androidx.compose.ui:ui-test-manifest")
}
```

build.gradle.kts (extrait)

Création des fichiers du projet

User.kt

```
package com.example.authapp.data

data class User(
    val fullName: String,
    val email: String,
    val password: String
)
```

app/data/User.kt

AuthViewModel.kt

```
package com.example.authapp

import androidx.lifecycle.ViewModel
import com.example.authapp.data.User
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow

class AuthViewModel : ViewModel() {
    private val _currentUser = MutableStateFlow<User?>(null)
    val currentUser: StateFlow<User?> = _currentUser.asStateFlow()

    private val registeredUsers = mutableListOf<User>()

    fun signUp(fullName: String, email: String, password: String):
       Boolean {
        if (registeredUsers.any { it.email == email }) return false
        val newUser = User(fullName, email, password)
        registeredUsers.add(newUser)
        _currentUser.value = newUser
        return true
    }

    fun login(email: String, password: String): Boolean {
        val user = registeredUsers.find { it.email == email && it.
           password == password }
        _currentUser.value = user
        return user != null
    }

    fun logout() {
        _currentUser.value = null
    }
}
```

app/AuthViewModel.kt

LoginScreen.kt

```
package com.example.authapp.screens

import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.input.PasswordVisualTransformation
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.unit.dp
import com.example.authapp.AuthViewModel

@Composable
fun LoginScreen(
    viewModel: AuthViewModel,
    onNavigateToSignUp: () -> Unit,
    onLoginSuccess: () -> Unit
) {
    var email by remember { mutableStateOf("") }
    var password by remember { mutableStateOf("") }
    var errorMessage by remember { mutableStateOf<String?>(null) }

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(32.dp),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text("Connexion", style = MaterialTheme.typography.
           headlineMedium)

        Spacer(Modifier.height(24.dp))

        OutlinedTextField(
            value = email,
            onValueChange = { email = it },
            label = { Text("Email") },
            modifier = Modifier.fillMaxWidth(),
            keyboardOptions = KeyboardOptions(keyboardType =
               KeyboardType.Email)
        )

        Spacer(Modifier.height(16.dp))

        OutlinedTextField(
            value = password,
            onValueChange = { password = it },
            label = { Text("Mot de passe") },
            modifier = Modifier.fillMaxWidth(),
            visualTransformation = PasswordVisualTransformation(),
```

```
            keyboardOptions = KeyboardOptions(keyboardType =
                KeyboardType.Password)
        )

        if (errorMessage != null) {
            Text(errorMessage!!, color = MaterialTheme.colorScheme.
                error)
        }

        Spacer(Modifier.height(24.dp))

        Button(
            onClick = {
                if (email.isBlank() || password.isBlank()) {
                    errorMessage = "Veuillez remplir tous les champs"
                } else if (viewModel.login(email, password)) {
                    onLoginSuccess()
                } else {
                    errorMessage = "Email ou mot de passe incorrect"
                }
            },
            modifier = Modifier.fillMaxWidth()
        ) {
            Text("Se connecter")
        }

        Spacer(Modifier.height(16.dp))

        TextButton(onClick = onNavigateToSignUp) {
            Text("Cr er un compte")
        }
    }
}
```

app/screens/LoginScreen.kt

SignUpScreen.kt

```
package com.example.authapp.screens

import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.input.PasswordVisualTransformation
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.unit.dp
import com.example.authapp.AuthViewModel
import java.util.regex.Pattern

@Composable
fun SignUpScreen(
```

```
    viewModel: AuthViewModel,
    onNavigateToLogin: () -> Unit,
    onSignUpSuccess: () -> Unit
) {
    var fullName by remember { mutableStateOf("") }
    var email by remember { mutableStateOf("") }
    var password by remember { mutableStateOf("") }
    var confirmPassword by remember { mutableStateOf("") }

    var fullNameError by remember { mutableStateOf<String?>(null) }
    var emailError by remember { mutableStateOf<String?>(null) }
    var passwordError by remember { mutableStateOf<String?>(null) }
    var confirmPasswordError by remember { mutableStateOf<String?>(null
        ) }

    fun validateForm(): Boolean {
        var isValid = true

        if (fullName.isBlank()) {
            fullNameError = "Le nom est obligatoire"
            isValid = false
        } else fullNameError = null

        val emailPattern = Pattern.compile(
            "^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,6}$"
        )
        if (email.isBlank()) {
            emailError = "L'email est obligatoire"
            isValid = false
        } else if (!emailPattern.matcher(email).matches()) {
            emailError = "Format d'email invalide"
            isValid = false
        } else emailError = null

        if (password.length < 6) {
            passwordError = "Le mot de passe doit contenir au moins 6
                caract res"
            isValid = false
        } else passwordError = null

        if (confirmPassword != password) {
            confirmPasswordError = "Les mots de passe ne correspondent
                pas"
            isValid = false
        } else confirmPasswordError = null

        return isValid
    }

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(32.dp),
```

```
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text("Inscription", style = MaterialTheme.typography.headlineMedium)

        Spacer(Modifier.height(24.dp))

        OutlinedTextField(
            value = fullName,
            onValueChange = { fullName = it },
            label = { Text("Nom complet") },
            modifier = Modifier.fillMaxWidth(),
            isError = fullNameError != null,
            supportingText = { if (fullNameError != null) Text(fullNameError!!) }
        )

        Spacer(Modifier.height(16.dp))

        OutlinedTextField(
            value = email,
            onValueChange = { email = it },
            label = { Text("Email") },
            modifier = Modifier.fillMaxWidth(),
            keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Email),
            isError = emailError != null,
            supportingText = { if (emailError != null) Text(emailError!!) }
        )

        Spacer(Modifier.height(16.dp))

        OutlinedTextField(
            value = password,
            onValueChange = { password = it },
            label = { Text("Mot de passe") },
            modifier = Modifier.fillMaxWidth(),
            visualTransformation = PasswordVisualTransformation(),
            keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Password),
            isError = passwordError != null,
            supportingText = { if (passwordError != null) Text(passwordError!!) }
        )

        Spacer(Modifier.height(16.dp))

        OutlinedTextField(
            value = confirmPassword,
            onValueChange = { confirmPassword = it },
            label = { Text("Confirmer le mot de passe") },
```

```
            modifier = Modifier.fillMaxWidth(),
            visualTransformation = PasswordVisualTransformation(),
            keyboardOptions = KeyboardOptions(keyboardType =
               KeyboardType.Password),
            isError = confirmPasswordError != null,
            supportingText = { if (confirmPasswordError != null) Text(
               confirmPasswordError!!) }
        )

        Spacer(Modifier.height(24.dp))

        Button(
            onClick = {
                if (validateForm()) {
                    if (viewModel.signUp(fullName, email, password)) {
                        onSignUpSuccess()
                    } else {
                        emailError = "Cet email est  d  j   utilis "
                    }
                }
            },
            modifier = Modifier.fillMaxWidth()
        ) {
            Text("S'inscrire")
        }

        Spacer(Modifier.height(16.dp))

        TextButton(onClick = onNavigateToLogin) {
            Text(" D  j   un compte ? Se connecter")
        }
    }
}
```

app/screens/SignUpScreen.kt

HomeScreen.kt

```
package com.example.authapp.screens

import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import com.example.authapp.data.User

@Composable
fun HomeScreen(
    currentUser: User?,
    onLogout: () -> Unit
) {
```

```
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(32.dp),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text("Bienvenue !", style = MaterialTheme.typography.
           headlineLarge)

        currentUser?.let {
            Spacer(Modifier.height(16.dp))
            Text("Bonjour, ${it.fullName} !", style = MaterialTheme.
               typography.headlineMedium)
            Spacer(Modifier.height(8.dp))
            Text("Email : ${it.email}", style = MaterialTheme.
               typography.bodyLarge)
        }

        Spacer(Modifier.height(32.dp))

        Button(
            onClick = onLogout,
            colors = ButtonDefaults.buttonColors(
                containerColor = MaterialTheme.colorScheme.error
            )
        ) {
            Text("Se d connecter")
        }
    }
}
```

app/screens/HomeScreen.kt

MainActivity.kt

```
package com.example.authapp

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.viewModels
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.rememberNavController
import com.example.authapp.screens.HomeScreen
import com.example.authapp.screens.LoginScreen
import com.example.authapp.screens.SignUpScreen
```

```

class MainActivity : ComponentActivity() {
    private val viewModel: AuthViewModel by viewModels()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            AuthAppTheme {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    AuthApp(viewModel = viewModel)
                }
            }
        }
    }
}

@Composable
fun AuthApp(viewModel: AuthViewModel) {
    val navController = rememberNavController()

    NavHost(navController = navController, startDestination = "login") {
        composable("login") {
            LoginScreen(
                viewModel = viewModel,
                onNavigateToSignUp = { navController.navigate("signup") },
                onLoginSuccess = { navController.navigate("home") }
            )
        }

        composable("signup") {
            SignUpScreen(
                viewModel = viewModel,
                onNavigateToLogin = { navController.navigate("login") },
                onSignUpSuccess = { navController.navigate("home") }
            )
        }

        composable("home") {
            HomeScreen(
                currentUser = viewModel.currentUser.value,
                onLogout = {
                    viewModel.logout()
                    navController.navigate("login") {
                        popUpTo("home") { inclusive = true }
                    }
                }
            )
```

```
        }
    }
}

@Composable
fun AuthAppTheme(content: @Composable () -> Unit) {
    MaterialTheme(content = content)
}
```

app/MainActivity.kt

Test de l'application

1. Lancer sur un émulateur ou un appareil réel.
2. Cliquer sur **Créer un compte**, remplir et valider.
3. Vérifier l'affichage sur l'écran d'accueil.
4. Cliquer sur **Se déconnecter**.
5. Tester la reconnexion avec les mêmes identifiants.